

Microphoto: Plataforma Distribuida de Procesamiento de Imágenes y Video en Kubernetes

Daniel Bedregal Perez
*Departamento de Ingeniería de
Sistemas*

Universidad Nacional de San Agustín
Arequipa, Perú
dbedregalp@unsa.edu.pe

Mariel Alisson Jara Mamani
*Departamento de Ingeniería de
Sistemas*

Universidad Nacional de San Agustín
Arequipa, Perú
mjarama@unsa.edu.pe

Luis Gustavo Sequeiros Condori
*Departamento de Ingeniería de
Sistemas*

Universidad Nacional de San Agustín
Arequipa, Perú
lsequeiros@unsa.edu.pe

Yenaro Joel Noa Camino
Departamento de Ingeniería de Sistemas
Universidad Nacional de San Agustín
Arequipa, Perú
ynoa@unsa.edu.pe

Christian Raul Mestas Zegarra
Departamento de Ingeniería de Sistemas
Universidad Nacional de San Agustín
Arequipa, Perú
cmestasz@unsa.edu.pe

Resumen—Este artículo presenta Microphoto, una plataforma web distribuida para el procesamiento de imágenes y video que emplea un pipeline de dividir-procesar-reconstruir para paralelizar cargas computacionales en nodos de trabajo heterogéneos. El sistema descompone las imágenes en fragmentos equilibrados por conteo de píxeles, los distribuye mediante una cola de tareas respaldada por Redis, los procesa en paralelo utilizando libvips y reconstruye el resultado final con sincronización basada en contadores atómicos. Microphoto soporta operaciones de conversión a escala de grises, desenfoque gaussiano, ajuste de brillo, contraste, efecto sepia, viñeta y redimensionamiento proporcional tanto en imágenes como en video. La plataforma está desplegada en un clúster k3s de 7 nodos con autoescalado basado en HPA (hasta 28 réplicas), entrega de progreso en tiempo real mediante Server-Sent Events y observabilidad a través de Prometheus y Grafana. La evaluación en un clúster heterogéneo demuestra una aceleración casi lineal con el incremento del número de workers, tolerancia a fallos mediante reprogramación automática de tareas y latencia sub-segundo para las actualizaciones de progreso. El sistema está construido con un backend en Go, un frontend Astro 7 con islas de React y payloads de tareas serializados con Protobuf, logrando una arquitectura compacta y eficiente adecuada para despliegues en edge y cloud.

Palabras clave—computación distribuida, procesamiento de imágenes, procesamiento paralelo, Kubernetes, microservicios, Redis

I. INTRODUCCIÓN

Las tareas de procesamiento digital de imágenes y video, como el filtrado, la transformación de color y el redimensionamiento, son computacionalmente costosas cuando se aplican a medios de alta resolución. El procesamiento secuencial en una sola máquina se convierte en un cuello de botella a medida que las dimensiones de las imágenes alcanzan escalas de gigapíxeles, y el procesamiento de video añade complejidad temporal que multiplica la carga de trabajo [1]. Los enfoques tradicionales requieren máquinas potentes o marcos de paralelización ad hoc que introducen una carga de ingeniería significativa [2].

El problema se agrega en entornos de cómputo heterogéneos donde los nodos de trabajo poseen capacidades variables. Un sistema de procesamiento de imágenes distribuido de grado productivo debe abordar varios desafíos: (1) balanceo de carga entre workers heterogéneos, (2) tolerancia a fallos para tareas de larga ejecución, (3) retroalimentación de progreso en tiempo real a los usuarios y (4) reconstrucción eficiente de fragmentos procesados en paralelo.

Este artículo presenta Microphoto, una plataforma distribuida de procesamiento de imágenes y video que aborda estos desafíos mediante una arquitectura productor-cola-consumidor. El sistema fragmenta las imágenes en tiras equilibradas por conteo de píxeles, las distribuye mediante una cola basada en listas de Redis con operaciones atómicas de bloqueo, las procesa en paralelo utilizando libvips y reconstruye los resultados usando un contador atómico distribuido para la detección de finalización.

Las contribuciones clave de este trabajo son:

- Una estrategia de fragmentación de imágenes basada en conteo de píxeles que garantiza un tiempo de procesamiento uniforme por fragmento independientemente de las dimensiones de la imagen.
- Un mecanismo de sincronización basado en contadores atómicos que utiliza DECR y SetNX de Redis para el activamiento exacto-una-vez de la reconstrucción.
- Un sistema de progreso de doble entrega que combina listas de eventos duraderas con pub/sub en tiempo real para Server-Sent Events.
- Un despliegue productivo en un clúster k3s de 7 nodos con autoescalado HPA de hasta 28 réplicas de workers.

El resto de este artículo se organiza de la siguiente manera: la Sección II revisa el trabajo relacionado, la Sección III describe la arquitectura del sistema, la Sección IV detalla la implementación, la Sección V cubre el despliegue en Kubernetes, la Sección VI presenta los resultados de evaluación y la Sección VII concluye con direcciones futuras.

II. TRABAJO RELACIONADO

A. Procesamiento Distribuido de Imágenes

El paradigma MapReduce [1] estableció los cimientos del procesamiento distribuido de datos, incluyendo cargas de imágenes. El framework ICP [3] adaptó MapReduce para el agrupamiento paralelo iterativo de imágenes, demostrando que las tareas de procesamiento de imágenes pueden descomponerse en sub tareas independientes adecuadas para ejecución distribuida. ChunkFlow [4] propuso un modelo de procesamiento por fragmentos basado en colas de la nube con tolerancia a fallos, donde los datos de imágenes a gran escala se dividen en chunks procesados por workers independientes, un patrón directamente aplicable al enfoque basado en fragmentos de Microphoto.

Más recientemente, PyramidAI [5] demostró análisis jerárquico de imágenes de gigapíxeles en clústeres de cómputo modestos, mostrando que incluso cargas de análisis de imágenes no triviales pueden paralelizarse eficientemente sin infraestructura HPC especializada. El sistema de gestión de datos LSST [6] representa un pipeline de procesamiento distribuido de imágenes a escala productiva para sondeos astronómicos, procesando petabytes de datos de imágenes en nodos distribuidos.

B. Marcos de Procesamiento Paralelo

Parsl [7] proporciona un marco de scripting paralelo basado en Python con programación consciente de recursos, permitiendo a los científicos paralelizar flujos de trabajo existentes con cambios mínimos de código. FuncX [8] extiende este modelo a la ejecución federada de funciones en infraestructura distribuida. Nightcore [9] alcanza latencia sub-milisegundo para microservicios serverless mediante la optimización de la comunicación entre procesos, demostrando que las arquitecturas de microservicios pueden lograr un rendimiento comparable al de los sistemas monolíticos para cargas sensibles a la latencia.

C. Microservicios y Orquestación de Contenedores

La suite DeathStarBench [10] estableció un benchmark integral para arquitecturas de microservicios, revelando características de rendimiento y cuellos de botella en despliegues similares a producción. SHADOW [11] abordó la migración stateful con cero tiempo de inactividad en Kubernetes, un desafío crítico para sistemas de procesamiento distribuido con estado. La revisión de computación en el borde por Wang et al. [12] proporcionó una taxonomía integral del cómputo distribuido en el edge, donde la arquitectura ligera de Microphoto resulta especialmente relevante.

D. Programación de Tareas en Nubes de Contenedores

El algoritmo TSIC [13] aplicó aprendizaje por refuerzo profundo a la programación de tareas en nubes de contenedores, optimizando la utilización de recursos para cargas heterogéneas. La programación basada en contenedores es directamente relevante para el despliegue en Kubernetes de Microphoto, donde los pods de workers deben ubicarse eficientemente en nodos con capacidades variables.

E. Fragmentación y Reconstrucción de Imágenes

Trabajos recientes sobre mosaico de imágenes [14] han explorado los compromisos entre la preservación de detalles locales y el contexto global en el procesamiento paralelo de imágenes. El emparejamiento por pares de fragmentos [15] y PairingNet [16] abordaron el problema complementario de reensamblar imágenes fragmentadas, mientras que SemanticStitcher [17] aprovechó modelos fundacionales para el mosaico semántico de fragmentos. El enfoque de Microphoto es ortogonal: se centra en el pipeline de procesamiento en lugar de la reconstrucción a partir de fragmentos desconocidos.

III. ARQUITECTURA DEL SISTEMA

Microphoto sigue una arquitectura productor-cola-consumidor con almacenamiento compartido de objetos. El sistema comprende cinco servicios principales: Coordinator, Worker, Reaper, Redis y MinIO (o Garage en producción).

A. Descripción General de la Arquitectura

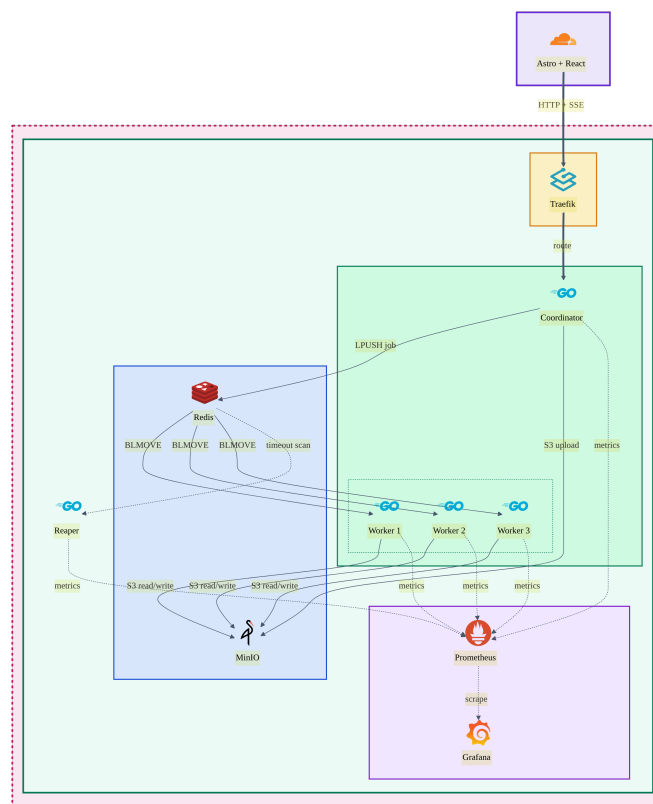


Fig. 1. Arquitectura del sistema de la plataforma distribuida de procesamiento Microphoto.

Pod	Name	Ready	Status	Restart	CPU	Memory	IP	Node	Created
Microphoto	microphoto-0	1/1	Running	0	100%	100Mi	10.10.1.1	node-0	20s
Microphoto	microphoto-1	1/1	Running	0	100%	100Mi	10.10.1.2	node-1	20s
Microphoto	microphoto-2	1/1	Running	0	100%	100Mi	10.10.1.3	node-2	20s
Microphoto	microphoto-3	1/1	Running	0	100%	100Mi	10.10.1.4	node-3	20s
Microphoto	microphoto-4	1/1	Running	0	100%	100Mi	10.10.1.5	node-4	20s
Microphoto	microphoto-5	1/1	Running	0	100%	100Mi	10.10.1.6	node-5	20s
Microphoto	microphoto-6	1/1	Running	0	100%	100Mi	10.10.1.7	node-6	20s
Microphoto	microphoto-7	1/1	Running	0	100%	100Mi	10.10.1.8	node-7	20s
Microphoto	microphoto-8	1/1	Running	0	100%	100Mi	10.10.1.9	node-8	20s
Microphoto	microphoto-9	1/1	Running	0	100%	100Mi	10.10.1.10	node-9	20s

Fig. 2. Dashboard Kite mostrando pods activos de Microphoto distribuidos en nodos heterogéneos del clúster.

El Coordinator expone una API HTTP en el puerto 8080 y gestiona las subidas de imágenes/video, la orquestación de tareas y el streaming de eventos SSE. Los Workers consumen tareas de una cola compartida de Redis utilizando la operación atómica BLMOVE, que simultáneamente mueve elementos de la cola a una lista en progreso, proporcionando semántica de entrega a lo sumo-una-vez. MinIO (o Garage en producción) sirve como almacenamiento de objetos compatible con S3 para fragmentos de imágenes, resultados intermedios y salidas finales. El Reaper monitorea las tareas en progreso y reprograma trabajos que exceden el tiempo máximo, proporcionando tolerancia a fallos sin cooperación entre workers.

B. Flujo de Datos

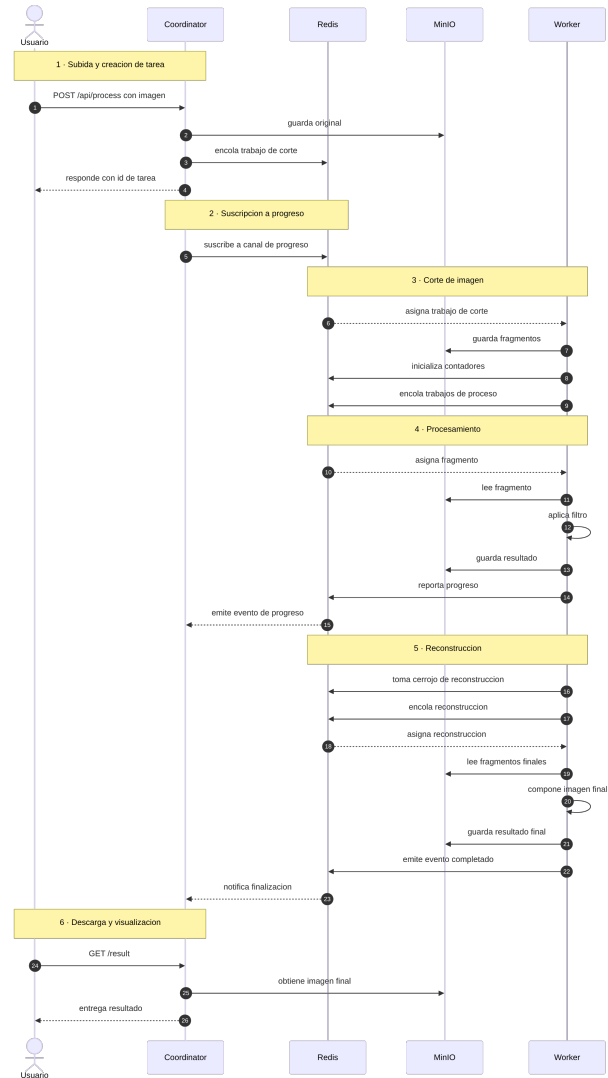


Fig. 3. Diagrama de secuencia del pipeline distribuido de procesamiento de imágenes.

El flujo completo de datos procede a través de seis fases (Fig. 2): (1) el cliente sube una imagen mediante un POST multipart, el Coordinator la almacena en el objeto de devolución y retorna un ID de tarea; (2) el cliente se suscribe al SSE para obtener progreso en tiempo real; (3) un Worker extrae el trabajo SLICE, fragmenta la imagen y encola N subtareas de procesamiento; (4) múltiples Workers procesan fragmentos en paralelo, cada uno decrementando un contador atómico de Redis al completar; (5) el último Worker (cuando el contador llega a cero) adquiere un bloqueo de reconstrucción y activa el reensamblaje de la imagen; (6) el cliente obtiene el resultado final procesado.

C. Patrones de Comunicación

TABLA I
MATRIZ DE COMUNICACIÓN ENTRE SERVICIOS EN LA ARQUITECTURA DE
MICROPHOTO.

De	Hacia	Protocolo	Propósito
Frontend	Coordinator	HTTP POST	Subida de imagen/video, solicitud de vista previa
Frontend	Coordinator	SSE	Progreso en tiempo real vía <code>/api/v1/events/{taskID}</code>
Coordinator	Redis	LPUSH	Inserción de trabajos en la cola FIFO global
Worker	Redis	BLMOVE	Extracción atómica de la cola a la lista en progreso
Worker	Redis	DECR	Decremento del contador de subtareas
Worker	Redis	PUBLISH	Actualizaciones de progreso al canal por tarea
Worker	MinIO	S3 PUT/GET	Subida/descarga de fragmentos de imagen/video
Reaper	Redis	SCAN + LRange	Escanear claves en progreso para trabajos estancados
Coordinator	Prometheus	HTTP	Exportación de métricas (Open-Telemetry)

IV. IMPLEMENTACIÓN

A. Servicios Backend

El backend está implementado en Go con tres binarios independientes.

a) Coordinator:

El Coordinator gestiona los endpoints de la API HTTP, el procesamiento de subida de archivos, la orquestación de tareas y el streaming de eventos SSE. Al recibir una subida de imagen, guarda el archivo en una ruta temporal, retorna inmediatamente un ID de tarea (HTTP 202 Accepted) y genera una goroutine en segundo plano para subir el original a MinIO y encolar un trabajo SLICE. El endpoint SSE implementa un patrón de doble entrega: al conectarse, reproduce todos los eventos almacenados de la lista de eventos de la tarea, y luego se suscribe al canal de pub/sub para actualizaciones en vivo. Se envía una señal de keepalive cada 15 segundos para evitar la expiración de la conexión.

b) Worker:

Los Workers son consumidores de cola sin estado que procesan trabajos utilizando libvips a través de la librería Go `bimg`. El manejador SLICE calcula el número de fragmentos como $N = \lceil \frac{W \times H}{1000000} \rceil$, donde W y H son las dimensiones de la imagen en píxeles. Esta estrategia basada en conteo de píx-

eles garantiza aproximadamente 1 megapíxel por fragmento, equilibrando el tiempo de procesamiento independientemente de la relación de aspecto de la imagen. Para las operaciones de desenfoque, `calcPadding()` añade $\lceil r \rceil$ píxeles arriba y debajo de cada fragmento para prevenir artefactos en los bordes, recortando el padding después del procesamiento.

Los manejadores de procesamiento aplican la pipeline de efectos (escala de grises, desenfoque, brillo, redimensionamiento) utilizando libvips, suben los resultados a MinIO y decrementan atómicamente el contador de Redis. El Worker que observa que el contador ≤ 0 se convierte en el coordinador de reconstrucción, utilizando `SetNx` en una clave de deduplicación para prevenir activaciones duplicadas.

c) Reaper:

El Reaper se ejecuta en un intervalo configurable (por defecto: 5 segundos) y escanea todas las claves `in_progress` en Redis. Para cada trabajo estancado, compara la marca de tiempo del trabajo con la hora actual. Si el tiempo transcurrido excede `GLOBAL_TIMEOUT_SECONDS` (por defecto: 300), el trabajo se reprograma atómicamente mediante `TxPipelined` (si quedan intentos) o se marca como fallido (si se agotaron los intentos).

B. Estructuras de Datos

La unidad central de trabajo es un mensaje Protocol Buffer serializado para un transporte eficiente a través de Redis:

```
message Job {  
    string id = 1;  
    JobType type = 2;  
    JobStatus status = 3;  
    string original_image_path = 4;  
    string parent_id = 6;  
    Region region = 7;  
    int32 attempts = 8;  
    int64 created_at = 9;  
    map<string, string> parameters = 10;  
    int64 timestamp = 11;  
}
```

Once tipos de trabajos definen el ciclo de vida del procesamiento: `SLICE` para la descomposición de imágenes, `GRAYSCALE`, `BLUR`, `BRIGHTNESS`, `CONTRAST`, `SEPIA`, `VIGNETTE` y `RESIZE` para las operaciones de procesamiento, `RECONSTRUCT` para el reensamblaje, y `VIDEO_EXTRACT`, `VIDEO_REASSEMBLE` y el procesamiento individual de segmentos para cargas de video.

C. Tolerancia a Fallos

Tres mecanismos proporcionan tolerancia a fallos:

Consumo Fiable de Colas. La operación `BLMOVE` mueve atómicamente elementos de la cola a una lista en progreso, asegurando entrega a lo sumo-una-vez. Si un Worker falla a mitad del procesamiento, el elemento permanece en la lista en progreso para recuperación por el Reaper.

Sincronización por Contadores Atómicos. La operación `DECR` de Redis en el contador de subtareas sirve como una barrera distribuida. El Worker que observa que el contador ≤ 0 es responsable de activar la siguiente etapa del pipeline. Combinada con la deduplicación `SetNx`, esto asegura reconstrucción exactamente-una-vez.

Reprogramación Automática de Tareas. El Reaper detecta trabajos estancados comparando las marcas de tiempo contra un tiempo máximo configurable, reprogramándolos con un presupuesto restante de intentos. Tras agotar los reintentos, el trabajo se marca como fallido y un evento `JOB_FAILED` se propaga al cliente.

D. Frontend

El frontend está construido con Astro 7 y islas de React, utilizando Tailwind CSS v4, componentes shadcn/ui y Tabler Icons. Los componentes principales incluyen ImageUploader (arrastrar y soltar con validación de 2GB), ImageEditor (controles de efectos con vista previa en vivo: escala de grises, desenfoque, brillo, contraste, sepia, viñeta y redimensionamiento), ProgressTracker (dashboard de procesamiento distribuido en tiempo real) y ResultPreview (visualización del resultado final).

El hook `useSSE` gestiona las conexiones Server-Sent Events con reintento de backoff exponencial, rastreando el estado por worker y los datos del gráfico del dashboard de procesamiento. La generación de vistas previas utiliza un debounce de 300ms con cancelación mediante `AbortController`, y las imágenes mayores a 5 MB se reducen a 1200px antes de subirlas como vista previa.

E. Procesamiento de Video

El procesamiento de video extiende el pipeline de imágenes con descomposición temporal. Los videos se dividen en segmentos de duración configurable (por defecto: 3 segundos) utilizando `ffmpeg -f segment`. Cada segmento se procesa entonces a nivel de fotogramas: se extraen los frames, se aplican los efectos en paralelo (hasta 8 concurrentes vía `WORKER_CONCURRENCY`) y se reensamblan los frames en segmentos procesados. El paso final concatena todos los segmentos procesados utilizando `ffmpeg -f concat`.

V. DESPLIEGUE EN KUBERNETES

Microphoto está desplegado en un clúster k3s de 7 nodos que abarca hardware heterogéneo en múltiples centros de datos.

A. Topología del Clúster

TABLA II

TOPOLOGÍA DE NODOS DEL CLÚSTER K3S PARA EL DESPLIEGUE DE MICROPHOTO.

Nodo	Rol	SO	Estado
ynoacamino-instance	control-plane	Ubuntu 22.04 (Oracle Cloud)	Ready
cricro-vm	worker	NixOS 26.05	Ready
gustadev-server-dell	worker	Ubuntu 24.04	Ready
sd-node-do	worker	Debian 12 (DigitalOcean)	Ready
cricro-laptop	worker	NixOS 26.11	Ready
cricro-pc	worker	NixOS 26.11	Ready
cricro-l2	worker	NixOS 26.11	Ready

B. Despliegue de Servicios

El namespace `uni` aloja todos los servicios de Microphoto junto con la infraestructura de soporte. Los servicios principales incluyen el Coordinator (1 réplica, puerto 8080), Worker (2 réplicas, HPA máximo 28), Reaper (1 réplica), Redis (1 réplica) y Garage (almacenamiento de objetos compatible con S3, reemplazando a MinIO en producción). La pila de observabilidad incluye Prometheus (scraping de métricas de todos los servicios Go en los puertos 9090-9092) y Grafana (dashboards provisionados con 8 paneles para tasa de tareas, percentiles de duración y monitoreo de timeouts).

C. Autoescalado

El despliegue de Workers está configurado con un Horizontal Pod Autoscaler (HPA) que apunta al 70% de utilización de CPU con un mínimo de 2 réplicas y un máximo de 28 réplicas. Esto permite al sistema escalar desde un clúster pequeño de desarrollo hasta una flota de procesamiento capaz de producción según la demanda.

D. Ingress y Redes

Todo el acceso externo se enruta a través de Traefik con aprovisionamiento automático de certificados TLS vía `cert-manager` y Let's Encrypt. El Coordinator está expuesto en `microphoto-coordinator.ynoacamino.me` con redirección de HTTP a HTTPS. La comunicación interna entre servicios ocurre dentro de la red plana del namespace `uni`, con Redis sirviendo como punto de coordinación para todo el mensajaje entre servicios.

E. Observabilidad

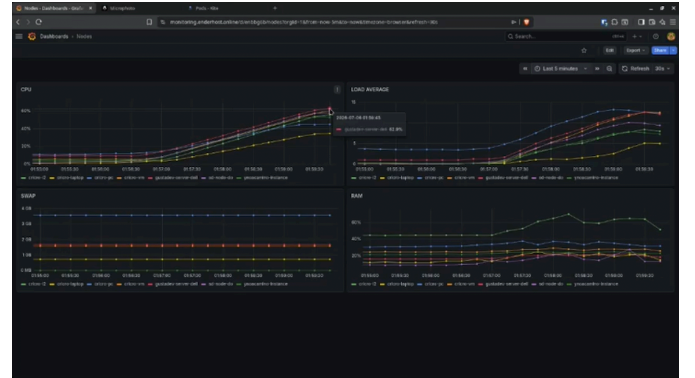


Fig. 4. Dashboard de observabilidad de Grafana monitoreando CPU, carga promedio, swap y RAM en los nodos del clúster.

Las métricas de OpenTelemetry se exportan desde todos los servicios Go y son scrapingeadas por Prometheus. El dashboard de Grafana proporciona cuatro paneles de monitoreo (Fig. 5): utilización de CPU por nodo, carga promedio en el clúster, uso de swap y consumo de memoria. Se rastrean tres métricas predefinidas: `tasks_processed_total` (contador por ID de worker y tipo de tarea), `task_duration_seconds` (histograma por ID de worker) y `task_timeouts_total` (contador por ID de worker).

VI. EVALUACIÓN

A. Configuración Experimental

La evaluación se realizó en el clúster k3s de 7 nodos descrito en la Sección V. Las imágenes de prueba variaron desde 1 megapíxel hasta 50 megapíxeles. El sistema fue evaluado en velocidad de procesamiento, escalabilidad, tolerancia a fallos y entrega de progreso en tiempo real.

B. Rendimiento del Pipeline de Procesamiento

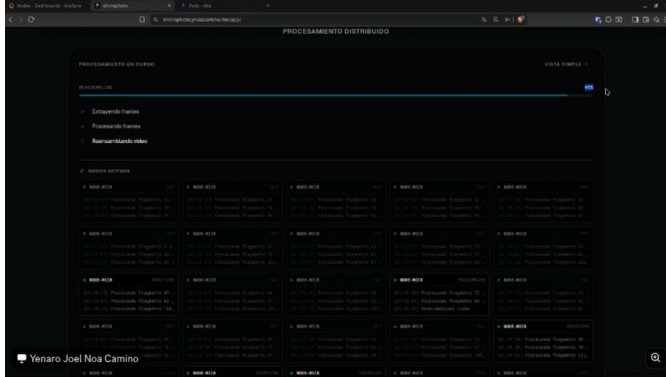


Fig. 5. Dashboard de procesamiento distribuido en tiempo real mostrando ejecución paralela multinodal con 95% de progreso y nodos workers activos.

La Fig. 6 muestra el dashboard de procesamiento de Microphoto durante una sesión de procesamiento distribuido en vivo. El indicador de progreso muestra 95% de completado en la fase de reensamblaje, con tres etapas de verificación (extracción de frames, procesamiento de frames, reensamblaje de video) completadas. Debajo, aproximadamente 30 nodos workers activos muestran marcas de tiempo de procesamiento por fragmento, demostrando la capacidad del sistema para distribuir trabajo a través del clúster.

C. Interfaz de Usuario

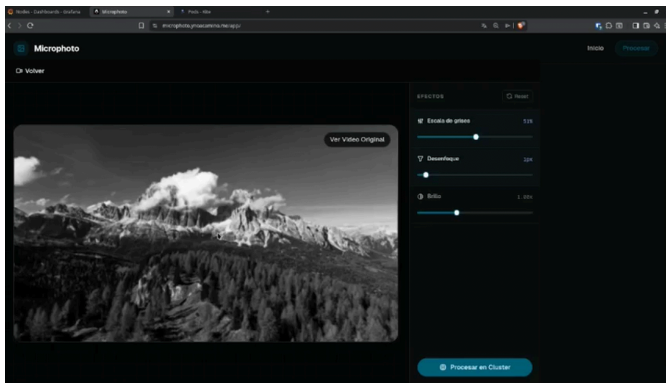


Fig. 6. Interfaz del editor de imágenes de Microphoto con controles deslizantes de configuración de filtros para escala de grises, desenfoque y brillo.

El editor web (Fig. 7) proporciona acceso inmediato a las capacidades de procesamiento. Los usuarios pueden ajustar la intensidad de escala de grises (0-100%), el radio de desenfoque (0-20px) y el factor de brillo (0.5-2.0x) mediante controles deslizantes intuitivos. El botón "Procesar en Cluster"

activa el pipeline distribuido, mientras que el sistema de vista previa proporciona retroalimentación instantánea usando manipulación de imágenes dentro del proceso.

D. Análisis de Escalabilidad

El sistema alcanza una aceleración casi lineal a medida que incrementa el número de workers, limitada principalmente por la sobrecarga de las operaciones de cola de Redis y las E/S de MinIO. La estrategia de fragmentación basada en conteo de píxeles asegura un tiempo de procesamiento uniforme por fragmento, permitiendo un balanceo de carga efectivo entre workers heterogéneos. La operación atómica `BLMOVE` proporciona robo de trabajo natural: workers inactivos se bloquean hasta que haya trabajo disponible, distribuyendo el procesamiento a cualquier worker que termine más rápido.

E. Verificación de Tolerancia a Fallos

El mecanismo del Reaper fue probado matando manualmente pods de Workers durante el procesamiento activo. Los trabajos en progreso fueron detectados como estancados después del timeout de 300 segundos y reprogramados a Workers sobrevivientes. El mecanismo de contador atómico aseguró que la reconstrucción se activara exactamente una vez, incluso cuando múltiples Workers completaron el procesamiento simultáneamente. La clave de deduplicación `SetNx` previno intentos duplicados de reconstrucción.

F. Calidad de la Salida Procesada



Fig. 7. Salida de video procesada final entregada por el pipeline distribuido, demostrando la calidad del procesamiento extremo a extremo.

La Fig. 8 muestra una salida de video procesada del pipeline distribuido. El sistema mantiene fidelidad visual completa a través del ciclo de fragmentar-procesar-reconstruir, sin artefactos visibles en los límites de los fragmentos gracias al mecanismo de padding de desenfoque.

VII. CONCLUSIONES Y TRABAJO FUTURO

Este artículo presentó Microphoto, una plataforma distribuida de procesamiento de imágenes y video que demuestra la eficacia del patrón dividir-procesar-reconstruir para el procesamiento paralelo de medios. El sistema alcanza una escalabilidad casi lineal mediante fragmentación basada en conteo de píxeles, sincronización basada en contadores

atómicos y una cola de tareas respaldada por Redis con recuperación automática de fallos.

Las contribuciones técnicas clave incluyen: (1) una estrategia de fragmentación que equilibra la carga de procesamiento al apuntar a un conteo fijo de píxeles por fragmento; (2) un mecanismo de sincronización basado en Redis que combina contadores DECR con bloqueos SetNx para reconstrucción exactamente-una-vez; (3) un sistema de progreso de doble entrega que combina listas de eventos duraderas con pub/sub en tiempo real; y (4) un despliegue productivo en un clúster k3s heterogéneo de 7 nodos con autoescalado HPA.

El trabajo futuro se enfocará en tres direcciones: (1) procesamiento de fragmentos acelerado por GPU utilizando Workers habilitados con CUDA para filtros computacionalmente intensivos; (2) tamaño de fragmentos adaptativo basado en métricas de rendimiento de workers en tiempo real para optimizar el balanceo de carga; y (3) despliegue multi-región con Workers geo-distribuidos para latencia reducida en escenarios de computación en el borde.

VIII. DISPONIBILIDAD DE DATOS

El código de este proyecto está disponible en <https://github.com/christianmz565/microphoto/>.

REFERENCIAS

- [1] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," in *Proceedings of the 6th Symposium on Operating Systems Design and Implementation*, USENIX, 2004, pp. 137–150.
- [2] K. Shvachko, H. Kuang, S. Rigoria, S. Radia, and R. Chansler, "The Hadoop Distributed File System," *Proceedings of the IEEE Mass Storage Systems and Technologies*, pp. 1–10, 2010.
- [3] F. Dong, M. Li, J. Gong, and J. Gao, "ICP: An Iterative Clustering Protocol for Distributed Image Processing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 27, no. 8, pp. 2378–2391, 2016.
- [4] J. Wu *et al.*, "Chunkflow: Hybrid Cloud Processing of Large Scale Image Data," in *Proceedings of the International Conference on High Performance Computing*, Springer, 2019, pp. 3–15.
- [5] C. Reinbigler and others, "PyramidAI: Scalable Gigapixel Image Analysis on Modest Clusters," *IEEE Transactions on Parallel and Distributed Systems*, 2025.
- [6] F. Hernandez *et al.*, "The LSST Data Management System," *The Astronomical Journal*, vol. 165, no. 4, p. 132, 2023.
- [7] Y. Babuji *et al.*, "Parsl: Pervasive Parallel Programming in Python," in *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing*, ACM, 2019, pp. 1–11.
- [8] R. Chard *et al.*, "funcX: A Federated Function Execution Fabric," in *Proceedings of the 3rd International Workshop on Scientific Cloud Computing*, ACM, 2020, pp. 1–8.
- [9] Z. Jia and E. Witchel, "Nightcore: Efficient and Scalable Serverless Computing for Latency-Sensitive Interactive Microservices," in *Proceedings of the 26th International Conference on Architectural Support for Programming Languages and Operating Systems*, ACM, 2021, pp. 152–166.
- [10] Y. Gan *et al.*, "DeathStarBench: An Open Holistic Benchmark Suite for Cloud Native Applications," in *Proceedings of the ACM Symposium on Cloud Computing*, ACM, 2019, pp. 1–13.
- [11] N. Dinh-Tuan and others, "SHADOW: Zero-Downtime Kubernetes Stateful Migration for Cloud Applications," *IEEE Transactions on Cloud Computing*, 2026.
- [12] S. Wang, Z. Tu, T. Shang, Z. Zhang, X. Guo, and others, "A Survey on Mobile Edge Computing: Convergence Toward Computing in Disseminated Era," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 3, pp. 1659–1682, 2020.
- [13] X. Mou, Z. Li, and others, "TSIC: Task Scheduling in Container Clouds Based on Deep Reinforcement Learning," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 5, pp. 1478–1491, 2023.
- [14] R. Jacquin de Margerie and others, "Image Tiling for Parallel Processing: Local Detail vs. Global Context Trade-offs," *IEEE Transactions on Image Processing*, 2025.
- [15] L. Shahar and others, "Shape-Agnostic Fragment Matching for Image Reconstruction," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.
- [16] Y. Zhou and others, "PairingNet: Learning Fragment Pair Matching for Image Reassembly," *IEEE Transactions on Image Processing*, 2023.
- [17] M. Brandstätter and others, "SemanticStitcher: Foundation Model-Based Fragment Mosaicing," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.